



Python Programming

We will be starting shortly

(Slides for today's lecture are on Scientia)



Quiz

What is a snake's favourite chart?

[answer given in lecture]

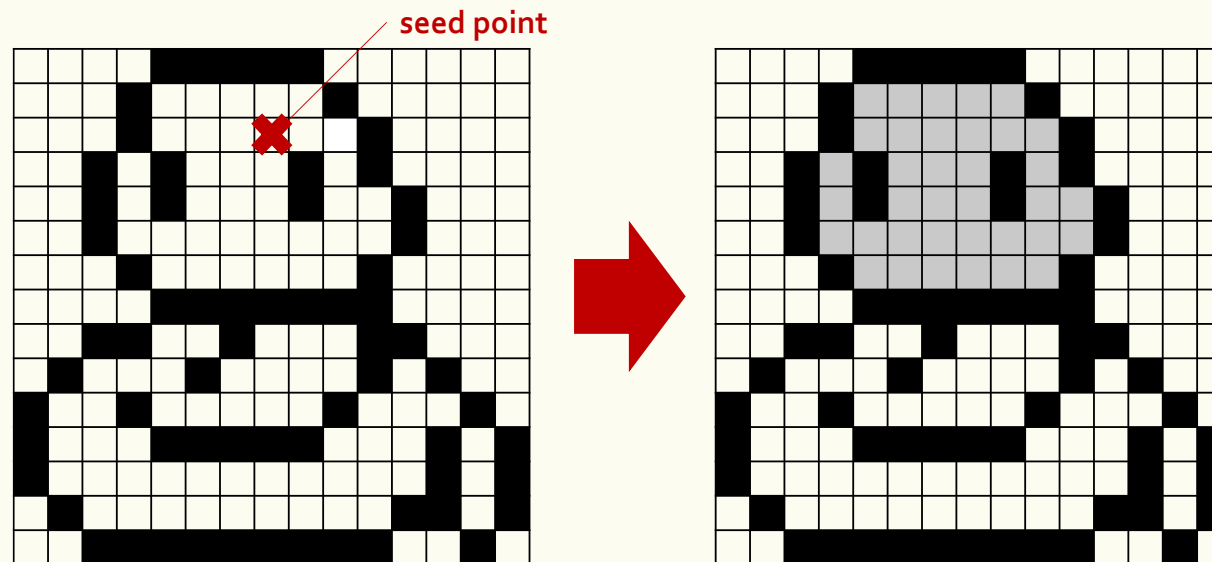


Coursework released!



Coursework 1

- Specifications on Scientia
- 3% of final module grade
- Deadline Monday 12th 7pm BST





Tasks

- Tasks
 - Complete the fill() function
 - Write some test cases (for valid and invalid cases)
 - Describe your algorithm (keep it brief!)
- Iterative or recursive solutions both accepted
 - But caution if you are using recursion (Python has a default recursion limit of 1000)



Gitlab Repo

- You should have already received your repo.



LabTS tests

- LabTS tests are just to make sure that your code works on the server!
 - You should still test your own code!
 - We will test your submission with another hidden set of tests



Submission

- Submit your commit hash via LabTS
- Submit 'readme.pdf' on Scientia
- Try submitting in the lab after this!



Tips

- I estimate 3-5 hours of work
- Think before you leap!



Previously...

Writing efficient code

- Improving the algorithm by reducing search space

 - Using efficient operations

 - Using optimal data structures



Previously...

```
filtered_items = []  
  
for item in items:  
    if item not in filtered_items:  
        filtered_items.append(item)
```

```
filtered_items = list(set(items))
```

```
filtered_items = list(dict(zip(items, [None]*len(items))))
```



Previously...

<filter_with_list>,	list size 10,	0.0008123 seconds
<filter_with_set>,	list size 10,	0.00037 seconds
<filter_with_dict>,	list size 10,	0.0007849 seconds
<filter_with_list>,	list size 100,	0.034291 seconds
<filter_with_set>,	list size 100,	0.0013613 seconds
<filter_with_dict>,	list size 100,	0.0031682 seconds
<filter_with_list>,	list size 600,	0.93456 seconds
<filter_with_set>,	list size 600,	0.011111 seconds
<filter_with_dict>,	list size 600,	0.021305 seconds
<filter_with_list>,	list size 4500,	11.232 seconds
<filter_with_set>,	list size 4500,	0.041099 seconds
<filter_with_dict>,	list size 4500,	0.12243 seconds
<filter_with_list>,	list size 30000,	91.623 seconds
<filter_with_set>,	list size 30000,	0.20946 seconds
<filter_with_dict>,	list size 30000,	0.79622 seconds



Previously...

- Is list comprehension faster? (Lesson 9)

```
s = []  
for item in items:  
    s.append(item)
```

```
S = [item for item in items]
```



Writing efficient code

that scales well with input size



Searching algorithms

because AI involves lots of searching!



```
numbers = [2, 95, 55, 72, 38, 46, 83, 51, 91, 17, 29]
```

42

```
42 in numbers
```

```
numbers.index(42)
```



Sequential/linear search

numbers = [2, 95, 55, 72, 38, 46, 83, 51, 91, 17, 29]



42

```
def sequential_search(query, items):  
    found = False  
    for item in items:  
        if item == query:  
            found = True  
  
    return found
```

```
sequential_search(42, numbers)
```

Time Complexity
Worst case: $O(n)$
Best case: $\Omega(n)$



Sequential/linear search

```
numbers = [2, 95, 55, 72, 38, 46, 83, 51, 91, 17, 29]
```

42

```
def sequential_search(query, items):  
    found = False  
    for item in items:  
        if item == query:  
            found = True  
            break  
    return found
```

```
sequential_search(42, numbers)
```

Time Complexity
Worst case: $O(n)$
Best case: $\Omega(1)$



Sequential/linear search

```
numbers = [2, 95, 55, 72, 38, 46, 83, 51, 91, 17, 29]
```

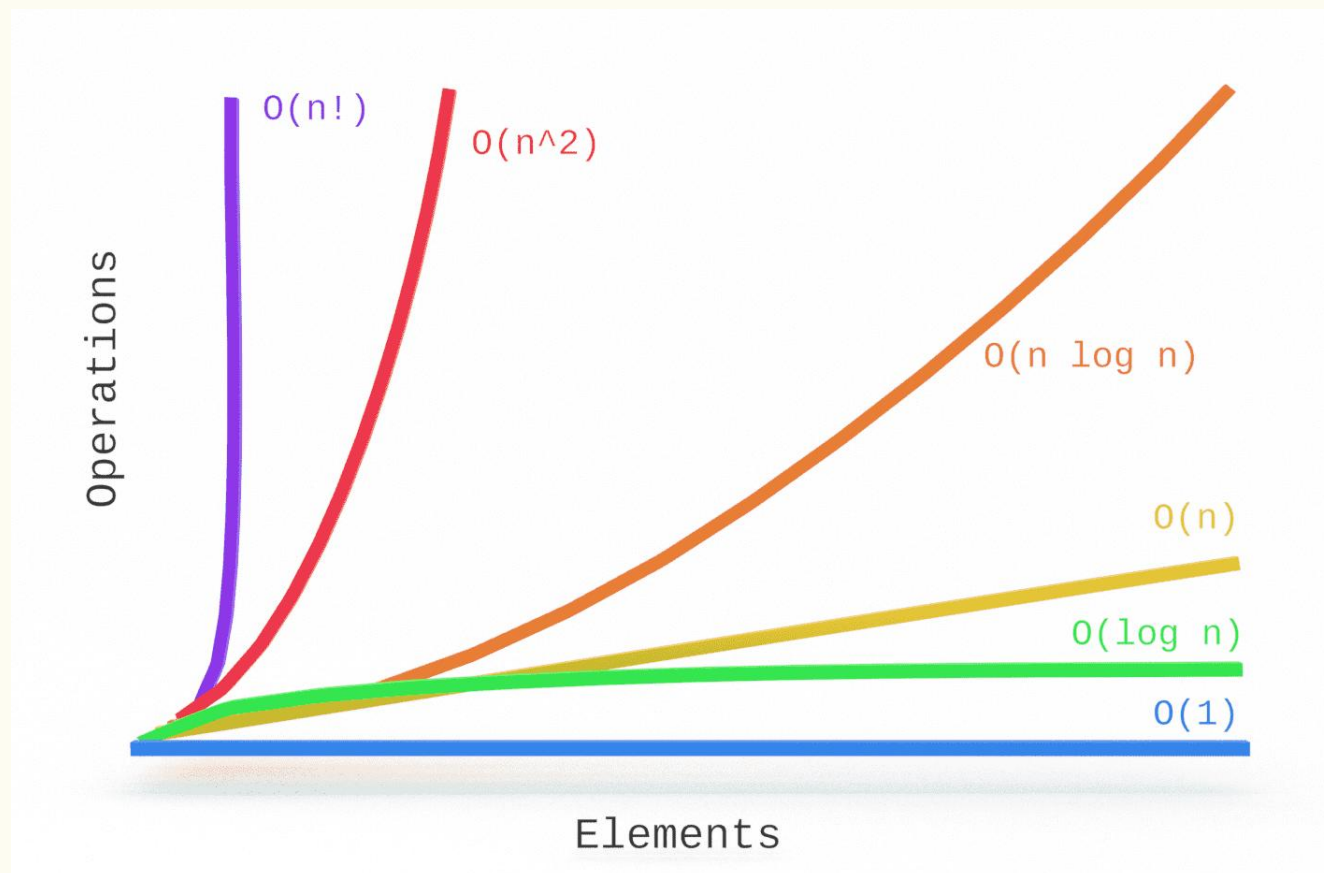
42

```
def sequential_search(query, items):  
    for (position, item) in enumerate(items):  
        if item == query:  
            return position  
  
    return -1
```

Time Complexity
Worst case: $O(n)$
Best case: $\Omega(1)$



Complexity... $O(n)$... what?



<https://superlative.guide/big-o-linear-time-complexity>

How many operations do you need to perform when input size is n ?

Asymptotic notation
Big-O notation

Constants dropped

Big-Omega, Big-Theta...



Can we do better than
sequential search?



Binary search

```
numbers = [2, 95, 55, 72, 38, 46, 83, 51, 91, 17, 29]
```



```
numbers = [2, 17, 29, 38, 46, 51, 55, 72, 83, 91, 95]
```

```
import bisect, random
```

```
numbers = []
```

```
for i in range(0, 11):
```

```
    number = random.randint(1, 100)
```

```
    bisect.insort(numbers, number)
```

```
    print(numbers)
```

To keep your list always sorted



Binary search

17

2	17	29	38	46	51	55	72	83	91	95
---	----	----	----	----	----	----	----	----	----	----





Binary search

17

2	17	29	38	46	51	55	72	83	91	95
---	----	----	----	----	----	----	----	----	----	----





Binary search

17

2	17	29	38	46	51	55	72	83	91	95
---	----	----	----	----	----	----	----	----	----	----





Binary search

17

2	17	29	38	46	51	55	72	83	91	95
---	-----------	----	----	----	----	----	----	----	----	----

↑ ↑



Binary search

17

2	17	29	38	46	51	55	72	83	91	95
---	----	----	----	----	----	----	----	----	----	----



Time Complexity

Best case: $\Omega(1)$

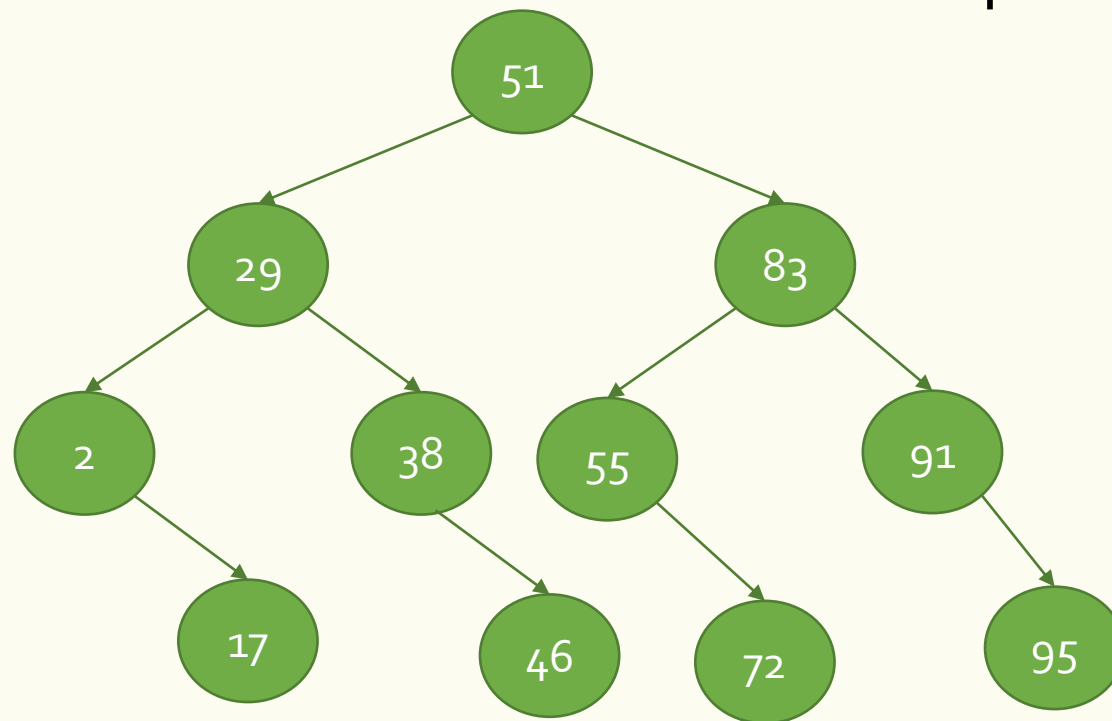
Worst case: $O(\log n)$



Binary tree

Depth of tree: $(\log_2 n) + 1$

$$(\log_2 11) = 3.46$$





2	17	29	38	46	51	55	72	83	91	95
---	----	----	----	----	----	----	----	----	----	----



Implement binary search?

Iterative

Recursive



2	17	29	38	46	51	55	72	83	91	95
---	----	----	----	----	----	----	----	----	----	----



Implement binary search?

Iterative



Iterative implementation

```
def binary_search(query, items):  
    left = 0  
    right = len(items) - 1  
  
    while left ≤ right:  
        middle = round(left + (right-left)/2)  
  
        if query == items[middle]:  
            return True  
        elif query < items[middle]:  
            right = middle - 1  
        else:  
            left = middle + 1  
  
    return False
```



2	17	29	38	46	51	55	72	83	91	95
---	----	----	----	----	----	----	----	----	----	----



Implement binary search?

Recursive



Recursive implementation

```
def binary_search(query, items):  
    if len(items) == 1:  
        return items[0] == query  
    else:  
        middle = round((len(items)-1)/2)  
        if query == items[middle]:  
            return True  
        elif query < items[middle]:  
            return binary_search(query, items[:middle])  
        else:  
            return binary_search(query, items[middle+1:])
```



Sorting algorithms

- Bubble sort
- Insertion sort
- Selection sort
- Merge sort
- Quick sort
- Heap sort
- ...



Summary

- Algorithm analysis
 - Concerned about time complexity w.r.t. input size
 - Best case, worst case
- Searching algorithms
 - Sequential search
 - Binary search
 - Iterative
 - Recursive



This and next week's schedule

Thu 4-6pm	Fri 11am-1pm
LAB 202/206	LAB 202/206

Tue 2-3pm	Tue 3-4pm	Thu 4-6pm	Fri 11am-1pm
LAB 219	LECTURE 144	LAB 202/206	LAB 202/206

Next week's lecture topic: Trees
CW1 due on Monday 7pm



Any (anonymous) feedback for us?

- <https://forms.gle/NbaFwtvT2xhWbeSW7>

